Capítulo 1 - Banco de dados e usuários de banco de dados

Um banco de dados é uma coleção de dados relacionados.

> 💡 **Dados** → fatos conhecidos que podem ser registrados e que possuem significado implícito

Um banco de dados tem as seguintes propriedades implícitas:

- Um banco de dados apresenta algum aspecto do mundo real, às vezes chamado de **minimundo** ou de **universo de discurso**. As mudanças no minimundo são refletidas no banco de dados.
- Um banco de dados é uma coleção logicamente coerente de dados com algum significado inerente. Uma variedade aleatória de dados não pode ser corretamente chamada de banco de dados.
- Um banco de dados é projetado, montado e preenchido com dados para uma finalidade especifica. Ele tem um grupo intencionado de usuários e algumas aplicações previamente concebidas nas quais esses usuários estão interessados.

Um banco de dados pode ter qualquer tamanho ou complexidade.

Um banco de dados pode ser mantido manualmente ou pode ser computadorizado.

Um **sistema de gerenciamento de banco de dados (SGBD)** é um sistema computadorizado que permite que os usuários criem e mantenham um banco de dados.

O SGBD é um sistema de software de uso geral que facilita o processo de *definição, construção, manipulação e compartilhamento de banco de dados.*

> 💡 A **definição** de um banco de dados envolve especificar os tipos de dados, as estruturas e restrições dos dados a serem armazenados. Ela é armazenada pelo SGBD na forma de um catálogo ou dicionário chamado de **metadados**.

💡 A **construção** do banco de dados é o processo de armazenar dados em algum meio de armazenamento pelo SGBD.

💡 A **manipulação** de um banco de dados inclui funções como consulta de dados, atualização de dados, geração de relatórios a partir dos dados.

💡 O **compartilhamento** permite que vários dados usuários e programas acessem o banco de dados simultaneamente.

Outras funções importantes fornecidas pelo SGBD incluem *proteção* do banco de dados e sua *manutenção* por um longo período de tempo.

💡 A **proteção** inclui *proteção do sistema* contra defeitos (ou falhas) de hardware ou software e *proteção de segurança* contra acesso não autorizado ou malicioso.

💡 **Sistema de banco de dados** → união do banco de dados com o software de SGBD.

Na abordagem de banco de dados, um único repositório mantém dados que são definidos uma vez e depois acessados por vários usuários, repetidamente por meio de consultas, transações e programas de aplicação.

Natureza de autodescrição de um sistema de banco de dados

Um característica fundamental na abordagem de banco de dados é que o sistema contém não apenas o próprio banco de dados, mas também uma definição ou descrição completa de sua estrutura e suas restrições.

A informação é armazenada no catálogo é chamada de **metadados** e descreve a estrutura do banco de dados principal. O catálogo é usado pelo software de SGBD e também pelos usuários do banco de dados que precisam de informações sobre a estrutura do banco de dados. Um pacote de software de SGBD de uso geral não é escrito para uma aplicação de banco de dados específica. O software de SGBD precisa

trabalhar de forma satisfatória com qualquer quantidade de aplicações de banco de dados, desde que a definição do banco de dados esteja armazenada no catálogo.

Isolamento entre programas e dados, e abstração de dados

No processamento de arquivos tradicional, a estrutura de arquivos de dados está embutida nos programas de aplicação, de modo que quaisquer mudanças na estrutura de um arquivo podem exigir mudar todos os programas que acessam esse arquivo. Ao contrário, os programas de acesso ao SGBD não exigem essas mudanças na maioria dos casos. A estrutura dos arquivos de dados é armazenada no catálogo do SGBD separadamente dos programas de acesso. Chamamos essa propriedade de **independência de dados do programa**.

A característica que permite a independência de dados do programa e independência da operação do programa é chamada de abstração de dados. Um SGBD oferece aos seus usuários uma **representação conceitual de dados** que não inclui muito dos detalhes de como os dados são armazenados ou como as operações são implementadas. Informalmente, um **modelo de dados**, é um tipo de abstração de dados usado para oferecer essa representação conceitual.

Suporte para múltiplas visões dos dados

Um banco de dados normalmente tem muitos usuários, cada um podendo exigir uma perspectiva ou uma visão diferente. Uma visão (ou view) pode ser um conjunto de banco de dados ou conter dados virtuais derivado dos arquivos do banco de dados, mas que não estão armazenados explicitamente. Um SGBD multiusuário, cujo os usuários tem uma série de aplicações distintas, precisa oferecer facilidades para definir múltipla visões.

Compartilhamento de dados e processamento de transação multiusuário

Um SGBD multiusuário precisa permitir que múltiplos usuários acessem o banco de dados ao mesmo tempo. Isso é essencial para que os dados de várias aplicações sejam integrados e mantidos em um único banco de dados. O SGBD precisa incluir o software de **controle de concorrência**, a fim de garantir que vários usuários tentando atualizar os mesmo dados façam isso de maneira controlada, de modo que o resultado das atualizações seja correto. Esses tipos de aplicações geralmente são chamados de aplicações de **processamento de transação on-line(OLTP)**. Um papel fundamental do software de SGBD multi usuário é garantir que as transações concorrentes operem de modo correto e eficiente.

💡 Um **transação** é um programa em execução ou processo que inclui um ou mais acessos ao banco de dados, como a leitura ou atualização dos registros. Cada transação deverá executar um acesso ao banco de dados, sendo logicamente correta para ser executada em sua totalidade, sem interferência de outras transações.

O SGBD precisa impor diversas propriedades da transação:

- **Isolamento**: garante que cada transação parecerá executar isoladamente das outras transações, embora centenas de transações possam estar sendo executadas simultaneamente.
- **Atomicidade**: garante que todas as operações em uma transação sejam executadas ou nenhuma seja.

Atores em cena

Administradores de banco de dados

Recursos: primário - banco de dados / secundários SGDB e software relacionado.

Administrador de banco de dados:

- responsável pela administração desse recurso.
- autorizar o acesso ao banco de dados.
- coordenar e monitorar seu uso.
- adquirir recursos de software e hardware conforme a necessidade.
- lidar com problemas de falhas de segurança e demora de resposta do sistema.

Projetista de banco de dados

São responsáveis por:

- Indicar os dados a serem armazenados
- escolheres estruturas apropriadas para representar e armazenar esses dados
- se comunicar com todos os usuários do banco de dados em potencial a fim de entender seus requisitos e criar um projeto que os atenda

Usuários finais

Suas funções:

- acesso ao banco de dados para consulta, atualização e geração de relatórios.

Categorias de usuários finais:

- **usuários finais causais**: utilizam uma interface de consulta sofisticada para especificar suas solicitações e normalmente são gerentes de nível intermediário ou alto, ou outros usuários ocasionais;
- **usuários finais iniciantes ou paramétricos**: consultar e atualizar o banco de dados constantemente, usando tipos padrão de consultas e atualização;
- **usuários finais sofisticados**: engenheiros, cientistas, analista de negócios e outros que estão profundamente familiarizados com as facilidades do SGDB a fim de implementar suas próprias aplicações, que atenderão ao seus requisitos complexos.
- **usuário isolado**: mantêm banco de dados pessoais usando pacote de programas prontos, que oferecem interfaces de fácil utilização, com base em menus ou em gráficos.

Analista de sistemas e programadores de aplicações (engenheiros de software)

Determinam os requisitos dos usuários finais, especialmente os usuários comuns ou paramétricos, e desenvolvem especificações para transações programadas que cumpram esses requisitos. Eles implementam essas especificações como programas, depois eles testam, depuram, documentam e mantêm essas transações programadas.

Trabalhadores dos bastidores

- Projetistas e implementadores de sistema de SGBD: projetam e implementam os módulos e interfaces do SGBD como um pacote de software.
- Desenvolvedores de ferramentas: projetam e implementam ferramentas - os pacotes de software que facilitam a modelagem e o projeto do banco de dados, o projeto do sistema do banco de dados e a melhoria do desempenho.
- Operadores e pessoal de manutenção: são responsáveis pela execução e manutenção real do ambiente de hardware e software para o sistema do banco de dados.

Vantagens de usar a abordagem de SGDB

Controlando a redundância

No desenvolvimento de software tradicional, utilizando processamento de arquivos, cada grupo de usuário mantém seus próprios arquivos para tratar de suas aplicações de processamento de dados. Dessa forma se corre o risco de aparecerem **redundâncias e dados inconsistentes**, uma vez que cada grupo de usuário atualiza sua própria tabela que, apesar de conterem informações compartilhadas, não compartilham as alterações entre si. Assim dados podem ser *armazenados com erro, de forma inconsistente, armazenados mais de uma vez e ocupar muito mais espaço*.

O ideal é que tenhamos um projeto de banco de dados que armazena cada item de dado lógico *em apenas um lugar*. Isso é conhecido como **normalização de dados**, e garante consistência e economiza espaço de armazenamento.

Porém, as vezes se torna necessário usar a **redundância controlada**, para melhorar o desempenho da consulta. Colocando todos os dados juntos não precisamos pesquisar vários arquivos para coletar esses dados. Isto é conhecido como **desnormalização**. Nesses casos o SGDB deverá ter a capacidade de controlar essa *redundância* a fim de proibir inconsistências entre os arquivos.

Restringindo o acesso não autorizado

Quando vários usuários compartilham um banco de dados grande, é provável que a maioria deles não seja autorizada a acessar todas as informações. Logo, o tipo de operação de acesso também deve ser controlado. Um SGBD deve oferecer um **subsistema de segurança e autorização** que o DBA utiliza para criar e gerenciar contas. Então, o SGDB deve impor essas restrições automaticamente.

Oferecendo armazenamento persistente para objetos do programa

Os bancos de dados podem ser usados para oferecer armazenamento persistente para objetos e estruturas de dados do programa. Esse é um dos principais motivos para usar os sistemas de banco de dados orientados a objeto. Algumas linguagens de programação com Java possuem structs e classe e em seu programa os dados são perdidos a não ser que, os dados sejam convertidos e gravados explicitamente em arquivos permanentes e, quando é necessário ler esses dados o programador precisa converter do formato do arquivo para o formato da variável. Os sistemas de banco de dados orientados a objeto são compatíveis com linguagens de programação como o Java, e o software de SGBD realiza automaticamente quaisquer conversões necessárias. Logo, um objeto complexo pode ser armazenado permanentemente em

um SGBD orientado a objeto. Esse objeto é considerado ***persistente***, por sobrevive ao término da execução e pode ser recuperado mais tarde diretamente por outro programa.

Oferecendo estruturas de armazenamento e técnicas de pesquisa para o processamento eficiente de consulta

Sistemas de banco de dados precisam oferecer capacidades para executar consultas e atualizações de modo eficiente. Como o banco de dados normalmente é armazenado em disco, o SGBD precisa oferecer estruturas de dados e técnicas de pesquisa especializadas para agilizar a busca dos registros desejados no disco. Arquivos auxiliares, chamados **índices**, são comumente utilizados para essa finalidade. Os índices normalmente são baseados em estruturas de dados em árvore ou estruturas de dados em hash, que são modificados de modo adequado para a pesquisa no disco.

O módulo de **processamento e otimização de consultas** do SGBD é responsável por escolher um plano de execução eficiente para cada consulta, com base nas estruturas de armazenamento existentes. A escolha de quais índices criar e manter faz parte do *projeto e ajuste do banco de dados físico*, que é uma das responsabilidades da equipe de DBAs.

Oferecendo backup e recuperação

Um SGBD precisa oferecer facilidades para recuperar-se de falhas de hardware ou software. O **subsistema de backup e recuperação** do SGBD e o responsável pela recuperação. O backup de disco também é necessário no caso de uma falha de disco catastrófica.

Oferecendo múltiplas interfaces ao usuário

Como muitos tipos de usuários com diversos níveis de conhecimento técnico utilizam um banco de dados, um SGBD deve oferecer uma série de interfaces do usuário. Estas incluem aplicativos para usuários móveis, linguagens de programação para programadores de aplicação, formulários e códigos de comando para usuários paramétricos, além de interfaces controladas por menu e interfaces de linguagem natural para usuários isolados.

Representando relacionamentos complexos entre dados

Um banco de dados pode incluir muitas variedades de dados que estão inter-relacionados de diversas maneiras. Um SGBD precisa ter a capacidade de representear uma série de relacionamentos complexos entre os dados, definir novos

relacionamentos à medida que eles surgem e recuperar e atualizar dados relacionados.

Impondo restrições de integridade

A maioria das aplicações de banco de dados possui certas **restrições de integridade** que devem ser mantidas para os dados. Um SGBD deve oferecer capacidades para definir e impor essas restrições. O tipo mais simples de restrição de integridade envolve especificar um tipo de dados para cada item de dados. Um tipo de restrição mais complexo, que ocorre com frequência, envolve especificar que um registro em um arquivo deve estar relacionado a registros em outros arquivos. Isso é conhecido como restrição de **integridade referencial**. Outro tipo de restrição especifica unicidade sobre valores de item de dados, como cada registro de curso deverá ter um valor único para Numero_disciplina. Isso é conhecido como uma restrição de **chave** ou **unicidade**. Essas restrições são derivadas do significado ou da **semântica** dos dados e do minimundo que eles representam. É responsabilidade dos projetistas de banco de dados identificar restrições de integridade durante o projeto. Algumas restrições podem ser especificadas ao SGBD e impostas automaticamente. Outras podem ter de ser verificadas por programas de atualização ou no momento da entrada de dados. Normalmente, para aplicações grandes, é comum chamar essas restrições de **regras de negócio**.

Regras que pertencem a um modelo de dados específico são chamadas **regras inerentes** ao modelo de dados.

Permitindo dedução e ações usando regras e gatilhos

Alguns sistemas de banco de dados oferecem capacidades para definir *regras de dedução* a fim de inferir novas informações com base nos fatos armazenados no banco de dados. Esses sistemas são chamados **sistemas de bancos de dados dedutivos**. Nos sistemas de banco de dados relacionais de hoje, é possível associar **gatilhos**(ou t**riggers**) a tabelas. Um gatilho é uma forma de regra ativada por atualizações na tabela, que resulta na realização de algumas operações adicionais em algumas outras tabelas, envio de mensagens e assim por diante. Procedimentos mais elaborados para impor regras são popularmente chamados de **procedimentos armazenados** (ou **stored procedures**); eles se tornam parte da definição geral de banco de dados e são chamados apropriadamente quando certas condições são atendidas. A funcionalidade mais poderosa é fornecida por **sistemas de banco de dados ativos**, que oferecem regras ativas que automaticamente iniciam ações quando ocorrem certos eventos e condições.

Implicações adicionais do uso da abordagem de banco de dados

- **Potencial para garantir padrões.** A abordagem de banco de dados permite que o DBA defina e garanta padrões entre os usuários em uma grande organização. Isso facilita a comunicação e a cooperação entre vários departamentos, projetos e usuários dentro da organização. Podem ser definidos padrões para nomes e formatos dos elementos de dados, formatos de exibição, estruturas de relatórios, terminologia e assim por diante.
- **Tempo reduzido para desenvolvimento de aplicações.** Uma característica importante de venda da abordagem de banco de dados é que o desenvolvimento de uma nova aplicação - como a recuperação de certos dados para a impressão de um novo relatório - leva muito pouco tempo; Projetar e implementar um grande banco de dados multiusuário do zero pode levar mais tempo que escrever um única aplicação de arquivo especializada. Porém, quando um banco de dados está pronto e funcionando, geralmente é preciso muito menos tempo para criar novas aplicações usando recursos do SGBD.
- **Flexibilidade.** Pode ser necessário mudar a estrutura de um banco de dados à medida que os requisitos mudam. Os SGBDs modernos permitem certos tipos de mudanças evolutivas na estrutura sem afetar os dados armazenados e os programas de aplicações existente.
- **Disponibilidade de informações atualizadas.** Um SGDB torna o banco de dados disponível a todos os usuários. Assim que a atualização de um usuário é aplicada, todos os outros usuários podem ver essa atualização imediatamente.
- **Economias de escala.** Isso permite que a organização interna invista em processadores, dispositivos de armazenamento ou mecanismos de rede mais poderosos, em vez de cada departamento ter de comprar seu próprio equipamento. Isso reduz os custos gerais de operação e gerenciamento.

Uma breve história das aplicações de banco de dados

Antigas aplicações de banco de dados usando sistemas hierárquicos e de rede

Muitas aplicações de banco de dados antigas mantinham registros em grandes organizações, como corporações, universidades, hospitais e bancos. Em muitas dessas aplicações, havia grandes quantidades de registros com estruturas semelhantes.

Um dos principais problemas com sistemas de bancos de dados antigos era a mistura de relacionamentos conceituais com o armazenamento e o posicionamento físico dos registros no disco. Logo, esses sistemas não ofereciam capacidades suficientes para abstração de dados e independência entre dados e programas. Por exemplo, os registros de notas de determinado aluno poderiam ser armazenados fisicamente próximos do registro do aluno. Embora isso fornecesse um acesso muito eficiente para as consultas e transações originais que o banco de dados foi projetado para lidar, não oferecia flexibilidade suficiente para acessar registros de modo eficiente quando novas consultas e transações fossem identificadas. Em particular, novas consultas que exigiam uma organização de armazenamento diferente para o processamento eficiente eram muito difíceis de implementar de modo eficaz. Também era muito trabalhoso reorganizar o banco de dados quando eram feitas mudanças nos requisitos da aplicação.

Outra deficiência dos sistemas antigos era que eles ofereciam apenas interfaces da linguagem de programação. Isso tronava a implementação de novas consultas e transações demorada e cara, pois novos programas tinha de ser escritos, testados e depurados. A maioria desses sistemas de banco de dados era implementada em computadores mainframe grandes e caros, começando em meados da década de 1960 e continuando pelos anos 1970 e 1980. Os primeiros tipos dos primeiros sistemas era baseados em três paradigmas principais: sistemas hierárquico, sistemas em modelo de rede e sistema de arquivos invertido.

Oferecendo abstração de dados e flexibilidade de aplicação com bancos de dados relacionais

Os bancos de dados relacionais foram propostos originalmente para separar o armazenamento físico dos dados de sua representação conceitual e para fornercer uma base matemática para a representação e a consulta dos dados. O modelo de dados relacional também introduziu linguagens de consulta de alto nível, que forneciam uma alternativa às interfaces de linguagem de programação, tornando muito mais rápido a escrita de novas consultas. Os sistemas relacionais visavam inicialmente às mesmas aplicações dos sistemas mais antigos, e forneciam flexibilidade para desenvolver novas consultas rapidamente e reorganizar o banco de dados à medida que os requisitos mudavam. Logo, *a abstração de dados e a independência entre dados e programas* foram muito melhoradas em comparação com os sistemas anteriores.

Os sistemas relacionais experimentais, desenvolvidos no final da década de 1970, e os sistemas de gerenciamento de banco de dados relacionais (SGBDR), introduzidos na década de 1980, eram muito lentos, pois não usavam ponteiros de

armazenamentos físico ou posicionamento de registro para acessar registros de dados relacionados. Com o desenvolvimento de novas técnicas de armazenamento e indexação e melhorias no processamento e otimização de consulta, seu desempenho melhorou. Por fim, os bancos de dados relacionais se tornaram o tipo de sistema de banco de dados dominante para aplicações tradicionais de banco de dados. Agora existem banco de dados relacionais para quase todos os tipos de computadores, desde os menores computadores pessoais até os grandes servidores.

Aplicações orientadas a objeto e a necessidade de bancos de dados mais complexos

O surgimento de linguagens de programação orientadas a objeto no final da década de 1980 e a necessidade de armazenar e compartilhar objetos complexos e estruturados levou ao desenvolvimento de bancos de dados orientados a objeto(BDOOs). Inicialmente, os BDOOs eram considerados como concorrentes dos bancos de dados relacionais, pois forneciam estruturas de dados mais gerais. Eles também incorporavam muitos dos paradigmas úteis orientados a objeto, como tipos de dados abstratos, encapsulamento de operações, herança e identidade de objeto. Porém, a complexidade do modelo e a falta de um padrão inicial contribuíram para seu uso limitado. Eles agora são usados principalmente em aplicações especializadas, como projetos de engenharia, publicações de multimídia e sistemas de manufatura. Apesar das expectativas de que eles terão grande impacto, sua penetração geral no mercado de produtos de banco de dados permanece baixa. Além disso, muitos conceitos de orientação a objetos foram incorporados nas versões mais recentes dos SGBDs relacionais, levando a sistemas de gerenciamento de banco de dados objeto-relacionais, conhecidos como SGBDORs.

Intercâmbio de dados na web para e-commerce usando XML

Grande parte da informação crítica sobre páginas web de e-comerce consiste em dados extraídos dinamicamente de SGBDs, como informações de voos, preços e disponibilidade de produtos. Diversas técnicas foram desenvolvidas para permitir o intercâmbio de dados extraídos na web para exibição em páginas. E eXtended Markup Language (XML) é um padrão para intercâmbio entre diversos tipos de bancos de dados e páginas web. A XML combina conceitos dos modelos usados nos sistemas de documentos com os conceitos de modelagem de banco de dados.

Estendendo as capacidades do banco de dados para novas aplicações

O sucesso dos sistemas de bancos de dados nas aplicações tradicionais encorajou os desenvolvedores de outros tipos de aplicações a tentarem utilizá-los. Essas aplicações tradicionalmente usavam suas próprias estruturas especializadas de arquivo e dados. Os sistemas de banco de dados agora oferecem extensões para dar melhor suporte aos requisitos especializados para algumas dessas aplicações.

Logo ficou claro que os sistemas relacionais básicos não eram muito adequados para muitas dessas aplicações, em geral por um ou mais dos seguintes motivos:

- Estruturas de dados mais complexas eram necessárias para modelar a aplicação do que a representação relacional simples.
- Novos tipos de dados eram necessários além dos tipos básicos numéricos e de cadeia de caracteres.
- Novas operações e construções de linguagem de consulta eram necessárias para manipular os novos tipos de dados.
- Novas estruturas de armazenamentos e indexação eram necessárias para a pesquisa eficiente sobre os novos tipos de dados.

Isso levou os desenvolvedores de SGBD a acrescentarem funcionalidades aos seus sistemas. Algumas funcionalidade eram de uso geral, como a incorporação de conceitos dos bancos de dados orientados a objeto aos sistemas relacionais. Outras funcionalidades eram de uso especial, na forma de módulos opcionais que poderiam ser usados para aplicações específicas.

Surgimento de sistemas de armazenamento big data e banco de dados NOSQL

Na primeira década do século XXI, a proliferação de aplicações e plataformas, como sites de redes sociais, grandes empresas de comércio eletrônico, índices de pesquisa na web e armazenamento/backup em nuvem, levou a um aumento na quantidade de dados armazenado em grandes bancos de dados e servidores robustos. Novos tipos de sistemas foram necessários para gerenciar esses imensos bancos de dados - sistemas que proporcionariam busca e recuperação rápidas, bem como armazenamento confiável e seguro de tipos não tradicionais de dados, como postagens de redes sociais e tuítes. Alguns dos requisitos desses novos sistemas não eram compatíveis com SGBDs relacionas SQL.

> 💡 SQL é uma padronização do modelo de dados e da linguagem para banco de dados relacionais

O termo NOSQL é geralmente interpretado como Não Apenas SQL, o que significa que, em sistemas que gerenciam grandes quantidades de dados, alguns deles são armazenados usando sistemas SQL, enquanto outros serão armazenados usando o NOSQL, dependendo dos requisitos da aplicação.

Quando não usar um SGBD

Apesar das vantagens de usar um SGBD, existem algumas situações em que um SGBD pode envolver custos adicionais desnecessários, que não aconteceriam no processamento de arquivos tradicional. Os custos adicionais do uso de um SGBD devem-se aos seguintes fatores:

- Alto investimento inicial em hardware, software e treinamento
- A generalidade que um SGBD oferecer para a definição e o processamento de dados.
- Esforço adicional para oferecer funções de segurança, controlo de concorrência, recuperação e integridade.

Portanto pode ser mais desejável usar arquivos comuns sob as seguintes circunstâncias:

- Aplicações de banco de dados simples e bem definidas, para as quais não se esperam muitas mudanças
- Requisitos rigorosos, de tempo real, para alguns programas de aplicação, que podem não ser atendidos em decorrência da sobrecarga de operações executadas pelo SGBD.
- Sistemas embarcados com capacidade de armazenamento limitada, nos quais um SGBD de uso geral não seria apropriado.
- Nenhum acesso de múltiplos usuários aos dados.

Perguntas de revisão

1. Defina os seguintes termos: dados, banco de dados, SGBD, sistema de banco de dados, catalogo de banco de dados, independência entre entrada de dados e programas, visão do usuário, DBA, usuário final, transação programada, sistema de banco de dados dedutivo, objeto persistente, metadados e aplicação de processamento de informação
2. Quais os quatro tipos principais de ações que envolvem banco de dados? Discuta cada tipo brevemente.

3. Discuta as principais características da abordagem de banco de dados e como ela difere dos sistemas de arquivos tradicionais.
4. Quais são as responsabilidades do DBA e dos projetistas de banco de dados?
5. Quais são os diferentes tipos de usuários finais do banco de dados? Discuta as principais atividades de cada um.
6. Discuta as capacidades que devem ser fornecidas pelo SGBD.
7. Discuta a diferença entre sistemas de banco de dados e sistemas de recuperação de informação.

Respostas:

1. Dados: fatos conhecidos que podem ser registrados e que possuem significado implícito;
   Banco de dados: uma coleção de dados relacionados;
   SGBD: é um sistema de software de uso geral que facilita o processo de definição, construção, manipulação e compartilhamento de banco de dados.
   Sistema de banco de dados: união do banco de dados com o software de SGBD;
   Catalogo de banco de dados: é a parte do sistema que contém uma definição ou descrição completa de sua estrutura e suas restrições;
   Independência entre entrada de dados e programas: propriedade que permite mudanças na estrutura dos dados sem exigir alterações nos programas de acesso.
   Visão do usuário: perspectiva personalizada dos dados, que pode ser um subconjunto ou dados derivados do banco de dados.
   DBA: É o administrador do banco de dados, responsável pela administração desse recurso, incluindo autorização, monitoramento e segurança;
   Usuário final: pessoa que acessa o banco de dados para consulta, atualização ou geração de relatórios;
   Transação programada: programa em execução que inclui um ou mais acessos ao banco de dados (leitura/atualização);
   Sistema de banco de dados dedutivo: sistema que permite definir regras de dedução para inferir novas informações a partir de fatos armazenados;
   Objeto persistente: objeto armazenado sem SGBD orientado a objeto que sobrevive ao término da execução do programa;
   Metadados: dados sobre os dados (estruturas e restrições), armazenados no catálogo.
   Aplicação de processamento de informação: aplicações que envolvem consultam atualização e geração de relatórios a partir de dados;

2. -Definição: especificar tipos de dados, estruturas e restrições dos dados;
   -Construção: processo de armazenar os dados em algum meio de armazenamento;
   -Manipulação: inclui consulta, atualização e geração de relatórios;
   -Compartilhamento: permite que múltiplos usuários e programas acessem o banco de dados simultaneamente;

3. Autodescrição: o banco de dados contém metadados (catálogo);
   Isolamento entre programas e dados (independência de dados);
   Abstração de dados: representação conceitual esconde detalhes de armazenamento;
   Múltiplas visões para diferentes usuários;
   Compartilhamento de dados e controle de concorrência;
   Controle de redundância e inconsistência;
   Restrição de acesso não autorizado;
   Backup e recuperação integrados;
   Suporte a múltiplas interfaces;

   Nos sistemas de arquivos tradicional, a estrutura de dados está embutida nos programas, gerando redundância, inconsistência e pouca flexibilidade.

4. DBA:
   -Autorizar acesso ao banco de dados;
   -Coordenar e monitorar o uso;
   -Adquirir recursos de software/hardware;
   -Lidar com falhas de segurança e resposta do sistema;
   Projetista de banco de dados:
   -Indicar quais dados armazenar;
   -Escolher estruturas apropriadas para representar e armazenar dados;
   -Comunicar-se com usuários para entender requisitos e criar um projeto que os atenda;

5. -Usuários finais casuais: acessam ocasionalmente com consultas sofisticadas (gerentes);
   -Usuários finais iniciantes ou paramétricos: consultam e atualizam constantemente com operações padrões;
   -Usuários finais sofisticados: engenheiros, cientistas; implementam suas próprias aplicações complexas;
   -Usuários isolados: mantêm bancos de dados pessoais com interfaces fáceis (menu, gráficos);

6. -Controle de redundância;
   -Restrição de acesso não autorizado;
   -Armazenamento persistente para objetos de programa;
   -Estruturas eficientes de pesquisa (índices);
   -Backup e recuperação;
   -Múltiplas interfaces de usuário;
   -Representação de relacionamentos complexos;
   -Imposição de restrições de integridade (chave, referencial, tipos);
   -Suporte a regras e gatilhos (triggers, stored procedures);
   -Controle de concorrência e atomicidade/isolamento de transações

7. Sistema de banco de dados: focam em dados estruturados, com modelo rígidos (relacional, orientado a objeto), suporte a transações, integridade, concorrência e consultas precisas;
   Sistema de recuperação de informação: lidam com dados semiestruturados e não estruturados (texto, documentos, páginas web), busca, por relevância, não exigem esquema rígido, e usam índices textuais.